FinCore

PRODUCT REQUIREMENTS DOCUMENT | DIGITAL WALLET | MVP 1.0

Версия: 1.0 • Дата: 14 октября 2026 • Статус: Baseline / готов к обсуждению

Назначение: продуктовая и техническая спецификация для разработки backend-first PET-проекта под Junior Python Backend portfolio.

1. Executive summary

FinCore — учебная платформа виртуальных денежных кошельков, позволяющая зарегистрированным пользователям создать CAD-кошелёк, получить тестовое пополнение, безопасно перевести средства другому пользователю и просматривать проведённые операции. Денежные движения фиксируются в неизменяемом double-entry ledger. Система не хранит и не перемещает настоящие деньги и не является банковским продуктом.

2. Цели, аудитория и метрики

Тип	Описание
Продуктовая цель	Надёжный API для виртуальных P2P-переводов в CAD с наблюдаемой финансовой историей.
Инженерная цель	Продемонстрировать ACID, SQL, конкурентность, идемпотентность, security, testing, CI/CD и деплой.
Пользователи	User: создаёт кошелёк и переводит деньги; Sandbox operator: запускает тестовые пополнения; System: ведёт ledger.
Success: correctness	0 нарушений денежных инвариантов в автоматизированных тестах, включая параллельные переводы.
Success: reproducibility	Проект запускается командой docker compose up, миграции и тесты проходят в CI.
Success: documentation	OpenAPI, README на английском, схема БД, ADR и сценарии проверки.

3. Scope MVP и исключения

Входит в MVP

- Email/password registration, login, refresh/logout и защищённые endpoints.
- Ровно один CAD-кошелёк для пользователя в MVP; схема допускает будущие кошельки других валют.
- Sandbox top-up с контролируемой авторизацией и бухгалтерским отражением.
- Синхронные P2P-переводы между активными CAD-кошельками.
- Immutable double-entry ledger и согласованная материальная проекция баланса.
- Idempotency key для всех POST, меняющих деньги.
- История переводов и операций кошелька с cursor pagination.
- Аудит чувствительных административных и денежных действий.
- Unit / integration / concurrency tests, Docker, Alembic, CI, деплой и документация.

Не входит в MVP

- Реальные банки, карты, платежи, Stripe, реальные финансовые обязательства и KYC.

- Hold/Release, reversals/refunds, fees, risk engine, recurring transfers, FX, statements export, реальные уведомления.
- Микросервисы, Kubernetes, ML antifraud, публичная админ-панель.
- Удаление проведённых ledger entries и редактирование завершённых денежных операций.

4. Термины и принципы

Термин	Определение
Wallet	Пользовательский виртуальный кошелёк в фиксированной валюте.
Posted balance	Проведённый остаток пользовательских обязательств; для liability account кредиты увеличивают, дебеты уменьшают.
Available balance	Для MVP равен posted balance, так как holds ещё нет.
Ledger account	Внутренний бухгалтерский счёт с типом ASSET / LIABILITY / REVENUE (в будущем).
Ledger transaction	Атомарная группа финансовых проводок единой бизнес-операции.
Ledger entry	Одна запись DEBIT либо CREDIT в определённой валюте и на определённом счёте.
Transfer	Бизнес-команда перевода денег между кошельками.
Idempotency	Повтор того же запроса с тем же ключом не должен создавать второе денежное движение.

5. User journeys и требования

US-01. Регистрация и авторизация

1. Гость передаёт email и пароль; сервис проверяет формат, уникальность и политику пароля.
2. Создаётся user со статусом ACTIVE; пароль хранится как password hash (Argon2id).
3. Пользователь входит, получает short-lived access token и rotateable refresh token.
4. Все операции кошелька проверяют token, user status и владение ресурсом.

US-02. Создание кошелька

5. Аутентифицированный пользователь отправляет POST /wallets с currency=CAD.
6. Сервис проверяет отсутствие кошелька CAD, создаёт Wallet, WalletBalance(0) и liability LedgerAccount.
7. Все три объекта создаются в одной БД-транзакции.
8. Повторный запрос на создание аналогичного кошелька не создаёт второй; conflict 409.

US-03. Sandbox top-up

9. Уполномоченный тестовый оператор создаёт запрос пополнения на сумму в центах и с idempotency key.
10. Проверяются статус кошелька, лимит, конфигурация окружения и уникальность ключа.
11. В одной транзакции создаются SandboxTopUp, LedgerTransaction, debit системного simulation funding asset и credit wallet liability.
12. Материальный баланс кошелька увеличивается; возвращается результат completed.

US-04. P2P transfer

13. Отправитель передаёт свой wallet ID, получателя, amount_minor и валюту, а также Idempotency-Key.
14. Проверяются сессия, принадлежность source_wallet, статусы, валюта, сумма, лимиты и различие кошельков.
15. Транзакция БД блокирует строки балансных проекций обеих сторон в детерминированном порядке.

16. Под блокировками ещё раз проверяется available balance и создаются Transfer, LedgerTransaction, 2 LedgerEntry.
17. Debit liability отправителя, credit liability получателя; обе проекции баланса обновляются атомарно.
18. Idempotency record фиксирует результат в той же транзакции, затем commit; API возвращает completed.

US-05. История

19. User открывает собственный кошелёк или перевод.
20. API проверяет доступ: только владелец source/target wallet может просматривать transfer (с ограничением раскрытия данных).
21. История сортируется по posted_at / id с cursor pagination, содержит входящие/исходящие суммы и ссылки на исходные операции.

6. Модель данных — MVP (полные сущности)

Конвенции: UUID — UUID v4 или UUID v7 с единым выбором; timestamps — TIMESTAMPTZ в UTC; amount_minor — BIGINT в минимальных единицах CAD; никакого float; все FK проиндексированы при необходимости. В примерах именование snake_case. Обязательные timestamps и audit-поля перечислены явно.

6.1 User

Агрегат идентификации. Не содержит денег и не владеет ledger напрямую.

Атрибут / тип	Ограничения и значение
id UUID PK	Глобальный идентификатор.
email VARCHAR(320) NOT NULL	Нормализованное значение; UNIQUE на normalized_email / citext.
password_hash TEXT NOT NULL	Argon2id; никогда не возвращать клиенту.
status VARCHAR(20) NOT NULL	ACTIVE / SUSPENDED / CLOSED.
created_at TIMESTAMPTZ NOT NULL	Создание.
updated_at TIMESTAMPTZ NOT NULL	Последнее обновление.
last_login_at TIMESTAMPTZ NULL	Последний успешный login.

6.2 RefreshSession

Серверный контроль refresh tokens, ротация и отзыв.

Атрибут / тип	Ограничения и значение
id UUID PK	Идентификатор сессии.
user_id UUID FK -> users.id	Владелец.
token_hash TEXT UNIQUE NOT NULL	Хэш случайного opaque refresh token, не сам токен.
created_at TIMESTAMPTZ	Время выдачи.
expires_at TIMESTAMPTZ	Время окончания.
revoked_at TIMESTAMPTZ NULL	Когда отозван.
replaced_by_session_id UUID NULL	Ссылка при ротации; опционально.

6.3 Wallet

Бизнес-сущность пользовательского кошелька; одна активная валюта на кошелёк.

Атрибут / тип	Ограничения и значение
id UUID PK	ID, видимый в API.
user_id UUID FK -> users.id	Владелец.
currency CHAR(3) NOT NULL	CAD в MVP; проверка разрешённых ISO codes в приложении/справочнике.
status VARCHAR(20) NOT NULL	ACTIVE / FROZEN / CLOSED.
created_at TIMESTAMPTZ	Создание.
updated_at TIMESTAMPTZ	Изменение.

UNIQUE(user_id,currency)	Один CAD-кошелёк на user в MVP, включая закрытые; политика повторного открытия отдельно.

6.4 WalletBalance

Транзакционно обновляемая проекция баланса; не первичный ledger.

Атрибут / тип	Ограничения и значение
wallet_id UUID PK FK -> wallets.id	Один к одному с wallet.
posted_minor BIGINT NOT NULL DEFAULT 0	CHECK >= 0 в MVP (без overdraft).
version BIGINT NOT NULL DEFAULT 0	Номер изменения для отладки/оптимистического контроля.
updated_at TIMESTAMPTZ	Последнее обновление.
available_minor	Не хранить: вычислять как posted_minor до появления Hold.

6.5 LedgerAccount

Бухгалтерский счёт; существует liability для каждого кошелька и один системный asset для sandbox funding.

Атрибут / тип	Ограничения и значение
id UUID PK	Идентификатор счёта.
wallet_id UUID FK NULL	Связь с кошельком для пользовательского LIABILITY, NULL для системного.
code VARCHAR(80) UNIQUE NOT NULL	Напр. WALLET:<uuid>:CAD или SYS:SIM_FUNDING:CAD.
account_type VARCHAR(20) NOT NULL	ASSET / LIABILITY; REVENUE зарезервировано для v1.1.
currency CHAR(3) NOT NULL	CAD в MVP.
status VARCHAR(20) NOT NULL	ACTIVE / CLOSED.
created_at TIMESTAMPTZ	Создание.
UNIQUE(wallet_id,account_type,currency)	Не допускать дублирования учётных счетов кошелька.

6.6 Transfer

Бизнес-результат перевода P2P, независимый от отдельных ledger entries.

Атрибут / тип	Ограничения и значение
id UUID PK	Уникальная операция.
sender_wallet_id UUID FK	Откуда.
receiver_wallet_id UUID FK	Куда.
amount_minor BIGINT NOT NULL	CHECK > 0; верхний лимит в конфигурации.
currency CHAR(3) NOT NULL	CAD.
status VARCHAR(20) NOT NULL	COMPLETED / REJECTED; устойчивых PENDING нет в MVP.
rejection_code VARCHAR(50) NULL	Только если REJECTED.
created_at TIMESTAMPTZ	Создание решения.
completed_at TIMESTAMPTZ NULL	Только для COMPLETED.
CHECK(sender_wallet_id <> receiver_wallet_id)	Различные кошельки.
Уникальность ledger link	Один успешный posting на transfer; enforcing ledger_transactions.transfer_id UNIQUE WHERE NOT NULL.

6.7 LedgerTransaction

Группа проводок для одной финансовой операции; один posted journal на transfer/topup.

Атрибут / тип	Ограничения и значение
id UUID PK	Идентификатор проводки.
transfer_id UUID FK NULL UNIQUE	Заполнено для transfer.
sandbox_topup_id UUID FK NULL UNIQUE	Заполнено для top-up.
kind VARCHAR(30) NOT NULL	P2P_TRANSFER / SANDBOX_TOPUP.
currency CHAR(3) NOT NULL	CAD; group currency.
posted_at TIMESTAMPTZ NOT NULL	Время бухгалтерского проведения.
reference VARCHAR(128) NULL	Человекочитаемый идентификатор.
CHECK exactly one source reference	Ровно transfer_id или sandbox_topup_id; реализация через CHECK.

6.8 LedgerEntry

Неизменяемая сторона проводки; каждая LedgerTransaction имеет >= 2 entries и balanced debit/credit.

Атрибут / тип	Ограничения и значение
id UUID PK	Идентификатор строки.
ledger_transaction_id UUID FK	Группа.
ledger_account_id UUID FK	Бухгалтерский счёт.
side VARCHAR(6) NOT NULL	DEBIT / CREDIT.
amount_minor BIGINT NOT NULL	CHECK > 0.
currency CHAR(3) NOT NULL	Должна совпадать с валютами group и account.
created_at TIMESTAMPTZ NOT NULL	Создание вместе с posting.
UNIQUE(ledger_transaction_id,ledger_account_id,side)	Для MVP одна запись на сторону/счёт; при усложнении ослабить.
IMMUTABILITY	DB privileges/trigger запрещают UPDATE и DELETE после проведения.

6.9 SandboxTopUp

Бизнес-документ тестового пополнения, связанный с симулируемым источником средств.

Атрибут / тип	Ограничения и значение
id UUID PK	ID операции.
wallet_id UUID FK	Получатель.
initiated_by_user_id UUID FK	Уполномоченный sandbox operator.
amount_minor BIGINT NOT NULL	CHECK > 0.
currency CHAR(3) NOT NULL	CAD.
status VARCHAR(20) NOT NULL	COMPLETED / REJECTED.
created_at TIMESTAMPTZ	Создание.
completed_at TIMESTAMPTZ NULL	Время commit/posting.

6.10 IdempotencyRecord

Уникальный запрос в контексте пользователя + endpoint; воспроизведение результата после сетевого сбоя.

Атрибут / тип	Ограничения и значение
id UUID PK	Идентификатор.
user_id UUID FK	Кто инициировал запрос.
operation_scope VARCHAR(80)	Напр. POST:/v1/transfers.
idempotency_key VARCHAR(255)	Ключ из заголовка.
request_hash CHAR(64) NOT NULL	SHA-256 канонического business payload + scope.
state VARCHAR(20) NOT NULL	IN_PROGRESS / COMPLETED, но промежуточная запись не должна оставаться навечно.
http_status SMALLINT NULL	Для replay.
response_body JSONB NULL	Снимок ответа без секретов.
resource_id UUID NULL	ID созданного Transfer/TopUp.
created_at, expires_at TIMESTAMPTZ	TTL хранения; политика MVP >= 24 h.
UNIQUE(user_id,operation_scope,idempotency_key)	Основной uniqueness index.

6.11 AuditLog

Append-only след действий (не замена ledger).

Атрибут / тип	Ограничения и значение
id UUID PK	Идентификатор события.
actor_user_id UUID NULL FK	User или системный actor.
action VARCHAR(80) NOT NULL	LOGIN_FAILED / WALLET_CREATED / TRANSFER_CREATED / TOPUP_CREATED и т.д.
entity_type VARCHAR(40) NOT NULL	USER / WALLET / TRANSFER / TOPUP / SESSION.
entity_id UUID NULL	ID объекта.
request_id UUID NULL	Correlation ID.
metadata JSONB NOT NULL DEFAULT {}	Только безопасные поля, без паролей и токенов.
created_at TIMESTAMPTZ	Создание.
IMMUTABILITY	События append-only, доступ ограничен.

7. Relationships / cardinality

Связь	Кардинальность	Правило
User -> Wallet	1:N	MVP фактически <= 1 CAD wallet.
Wallet -> WalletBalance	1:1	Баланс создаётся в одной транзакции с wallet.
Wallet -> LedgerAccount	1:N	В MVP ровно один liability account.
Wallet -> Transfer	1:N sender + 1:N receiver	Self-transfer запрещён.
Transfer -> LedgerTransaction	1:0..1	Для COMPLETED обязательно ровно 1.
SandboxTopUp -> LedgerTransaction	1:0..1	Для COMPLETED обязательно ровно 1.
LedgerTransaction -> LedgerEntry	1:N (N>=2)	Суммы debit/credit равны в одной валюте.
LedgerAccount -> LedgerEntry	1:N	Учётная история.
User -> RefreshSession	1:N	Ротация/отзыв refresh.
User -> IdempotencyRecord	1:N	Уникальность по user + scope + key.

8. Financial accounting model

8.1 Accounting convention

Денежные остатки клиентов учитываются на LIABILITY-счетах: увеличение обязательства — CREDIT, уменьшение — DEBIT. Виртуальное тестовое фондирование учитывается на системном ASSET-счёте: увеличение актива — DEBIT. Отрицательный баланс клиента не допускается. Для wallet liability posted_balance = SUM(CREDIT) - SUM(DEBIT) по проведённым entries.

8.2 Example P2P transfer CAD 100.00

Ledger account	Debit (cents)	Credit (cents)
Alice liability	10 000	0
Bob liability	0	10 000
TOTAL	10 000	10 000

8.3 Example sandbox top-up CAD 500.00

Ledger account	Debit (cents)	Credit (cents)
SYS:SIM_FUNDING:CAD (asset)	50 000	0
Alice liability	0	50 000
TOTAL	50 000	50 000

8.4 Mandatory invariants

INV-01 — Для каждой LedgerTransaction SUM(debits)=SUM(credits) в одинаковой валюте.

INV-02 — Каждый LedgerEntry ссылается на LedgerAccount той же валюты и на LedgerTransaction той же валюты.

INV-03 — Проведённые LedgerEntry / LedgerTransaction невозможно изменить или удалить.

INV-04 — WalletBalance.posted_minor равен суммы credit-minus-debit для соответствующего liability account.

INV-05 — WalletBalance.posted_minor >= 0 для каждого пользовательского кошелька.

INV-06 — Transfer COMPLETED имеет ровно один posted LedgerTransaction; REJECTED не имеет postings.

INV-07 — Нельзя наблюдать половину перевода: дебет, кредит, изменение обеих проекций и результат идемпотентности commit/rollback вместе.

INV-08 — Все amounts положительные BIGINT в центах; применены границы против integer overflow.

INV-09 — История финансовой операции имеет стабильные ID и timestamps; ручная правка запрещена.

INV-10 — Суммарное денежное изменение всех пользовательских кошельков от P2P переводов равно нулю.

Проверка сбалансированности — не обычный row CHECK: нужна единая функция posting с правами доступа и/или deferred constraint trigger к commit. Приложение не должно иметь возможность записать одну ledger entry без второй и завершить commit.

9. Concurrency, transaction and idempotency design

9.1 Transfer transaction boundary

22. Сделать быстрые предварительные проверки вне транзакции, но не считать их окончательными.
23. BEGIN; атомарно захватить/создать IdempotencyRecord по (user,scope,key); для конфликтующего in-progress запроса ждать результата либо вернуть retryable conflict.
24. Подтвердить request_hash. Если ключ существует и hash совпадает с COMPLETED, вернуть ранее сохранённый ответ. Если hash отличается — HTTP 409.
25. SELECT FOR UPDATE обеих WalletBalance строк в возрастающем wallet_id, чтобы избежать AB-BA deadlock.
26. Повторно проверить статус обоих Wallet, владение source, валюту и posted_minor >= amount_minor.
27. Создать Transfer(COMPLETED), LedgerTransaction, 2 LedgerEntry; синхронно пересчитать/обновить оба posted_minor.
28. Проверить balanced posting, неотрицательность балансов и согласованность ledger + projection.
29. Сохранить response/status в IdempotencyRecord и commit; вернуть HTTP 201.

Если business validation отклонена: не создавать бухгалтерских проводок; можно сохранить REJECTED business event/response в idempotency context для стабильного replay. При transient DB error выполнять rollback и повторять ограниченное число раз с backoff; не выполнять два независимо коммитящих сервиса.

9.2 Parallel transfer edge cases

- 100 CAD у Alice: два одновременных перевода по 80 CAD — только один completed.
- Alice -> Bob и Bob -> Alice одновременно — одинаковая последовательность блокировок, отсутствие deadlock в нормальном сценарии.
- Один ключ отправлен параллельно два раза — только один posted ledger transaction.
- Потеря HTTP-ответа после commit — replay возвращает исходный результат.
- Повтор с тем же ключом, но другой суммой/получателем — 409, без изменений балансов.
- Ошибка после первой ledger entry, до второй — rollback всех изменений.
- User пытается потратить чужой wallet — 403/404 без утечки чужого баланса.

10. REST API contract v1

Base path: /api/v1. JSON; даты ISO 8601 UTC; суммы — amount_minor. Correlation header X-Request-ID. Auth через Bearer access token. Все money-changing POST требуют Idempotency-Key; sandbox topup доступен только sandbox role.

Метод	Endpoint	Auth	Success
POST	/auth/register	Public	201 User
POST	/auth/login	Public	200 token pair
POST	/auth/refresh	Refresh token	200 rotated pair
POST	/auth/logout	User	204

GET	/users/me	User	200 UserProfile
POST	/wallets	User	201 Wallet
GET	/wallets	User	200 []
GET	/wallets/{wallet_id}	Owner	200 Wallet
GET	/wallets/{wallet_id}/balance	Owner	200 Balance
GET	/wallets/{wallet_id}/ transactions	Owner	200 cursor page
POST	/transfers	Owner/source	201 Transfer
GET	/transfers	User	200 cursor page
GET	/transfers/{transfer_id}	Sender/receiver	200 Transfer
POST	/sandbox/top-ups	Sandbox operator	201 SandboxTopUp
GET	/health/live	Public	200
GET	/health/ready	Internal monitoring	200/503

10.1 Request and response examples

POST /transfers | Idempotency-Key: 826d3e27-... | Authorization: Bearer <token>

{"source_wallet_id":"<uuid>","destination_wallet_id":"<uuid>","amount_minor":10000,"currency":"CAD"}

201:
{"id":"<uuid>","status":"COMPLETED","amount_minor":10000,"currency":"CAD","source_wallet_id":"<uuid>","destination_wallet_id":"<uuid>","created_at":"2026-10-14T12:00:00Z","completed_at":"2026-10-14T12:00:00Z"}

GET /wallets/{id}/balance →
{"wallet_id":"<uuid>","currency":"CAD","posted_minor":40000,"available_minor":40000,"as_of":"2026-10-14T12:00:00Z"}

GET /wallets/{id}/transactions?limit=20&cursor=<opaque> → {"items":[{"id":"<ledger_transaction_id>","direction":"OUTGOING","amount_minor":10000,"type":"P2P_TRANSFER","posted_at":"..."}],"next_cursor":null}

10.2 Error model

Uniform shape: {"error":{"code":"INSUFFICIENT_FUNDS","message":"Insufficient available balance","request_id":"<uuid>","details":{}}}. Не возвращать stack traces и внутренние SQL-сообщения.

HTTP	Code	Когда
400	INVALID_AMOUNT / INVALID_CURRENCY	Некорректные параметры.
401	UNAUTHENTICATED	Нет/некорректный токен.
403	FORBIDDEN	Нет разрешения.
404	NOT_FOUND	Ресурс не существует или скрыт из-за ACL.
409	WALLET_EXISTS / IDEMPOTENCY_CONFLICT	Конфликт состояния/ключа.
422	INSUFFICIENT_FUNDS / WALLET_INACTIVE	Бизнес-отклонение; стабильно задокументировать код.
429	RATE_LIMITED	Слишком много запросов.
503	DEPENDENCY_UNAVAILABLE	Нет критической зависимости; денег не изменено.

11. Security and privacy requirements

- Passwords: Argon2id + password policy; never log password/token.
- JWT access tokens: short TTL; refresh tokens: opaque random, hash at rest, rotation and revocation.
- Object-level authorization (source wallet owned by caller; target wallet existence disclosure controlled).
- Sandbox topups: no general public self-mint; use separate sandbox operator credential/role + configurable limits; disable outside demo/test environment.

- Request body strict schemas, maximum lengths, amount limits, rate limiting; no SQL text concatenation.
- TLS for deployed API; secrets in environment/secret store; no secrets in repository.
- Minimize PII; use synthetic demo users; log sanitized metadata only.
- Database role permissions: API role cannot UPDATE/DELETE posted ledger entries, migrations use separate credentials.
- Sensitive actions logged with request_id; protect audit logs against regular-user read.
- Token replay, invalid credentials, resource enumeration and race conditions tested.

12. Non-functional requirements (targets)

Area	MVP target / acceptance
Consistency	Financial invariants hold in every test and after forced rollback.
Latency	Local p95 simple reads < 200ms and transfers < 500ms under modest test load; benchmark, do not promise SLA.
Concurrency	100 concurrent transfer requests with controlled funds; no double spend.
Availability	Health/readiness endpoints; graceful DB failure without partial writes.
Maintainability	Modular monolith; boundaries users, wallets, transfers, ledger, sandbox, auth.
Observability	Structured JSON logs, request_id, transfer_id; counters for success/rejection/errors.
Pagination	Cursor-based pagination; bounded page size max 100.
Deployability	Docker Compose local, migrations, GitHub Actions, documented cloud deploy.
Testability	Tests run against real PostgreSQL container for transaction semantics; unit tests for validation.

13. Suggested backend architecture

Modular monolith: app/modules/auth, users, wallets, transfers, ledger, sandbox; app/core/database, config, security; app/shared/errors, types. Presentation layer (FastAPI routers) → application services (transaction orchestration) → repositories/DB models. Ledger posting exposed as a controlled domain operation. No independent commit in repositories.

Infrastructure: FastAPI + Pydantic v2 + SQLAlchemy 2.x + psycopg/asyncpg (one consistent stack) + Alembic + PostgreSQL + Redis (non-financial rate limiting) + Pytest + Ruff + mypy optional + Docker Compose + GitHub Actions. Redis failure must not corrupt monetary state.

14. Test plan / acceptance cases

ID	Test	Expected
ACC-01	Register, login, refresh, logout	Credentials hashed; invalid/expired sessions denied.
ACC-02	Create CAD wallet twice	Exactly one Wallet, Balance and liability account.
ACC-03	Top-up 500 CAD	Credit wallet liability, debit funding asset, posted=50000.
ACC-04	Transfer 100 CAD Alice->Bob	Alice -10000, Bob +10000, ledger balances per transaction.
ACC-05	Insufficient balance	No posting and no balance changes.
ACC-06	Two concurrent spends 80 from 100	At most one successful transfer.
ACC-07	Same idempotency key replay	Same recorded response; one posting.
ACC-08	Same key different payload	409 and no second posting.
ACC-09	Forced error after debit before credit	Everything rolled back.

ACC-10	Cross-account resource access	No balances/transfer details leaked.
ACC-11	Ledger reconciliation	Stored projection equals computed ledger balance.
ACC-12	Pagination and filtering	Stable ordering, no duplicates/misses in fixture set.
ACC-13	Frozen wallet	No sending/receiving according to documented policy.
ACC-14	100 concurrent randomized transfers	All balances >=0, ledger balanced, aggregate user funds unchanged.

15. Delivery milestones and definition of done

Milestone	Deliverable
M1 — Foundation	Project structure, Postgres, models/migrations, auth, test infra.
M2 — Wallets	User CAD wallet, balance projection, liability accounts.
M3 — Ledger	Posting service, sandbox funding, invariant validation and reconciliation.
M4 — Transfers	P2P, locks, idempotency, concurrent integration tests.
M5 — Portfolio release	OpenAPI, README English, diagrams, CI, Docker, deploy, demo, test report.

MVP Definition of Done: все acceptance tests зелёные на PostgreSQL; no double spend under concurrency; money postings immutable; replay semantics документированы; публичное демо использует только виртуальные средства; есть инструкция запуска, seed data и краткая архитектурная запись решений.

16. Post-MVP roadmap — согласованные функции

Версия	Фича	Новые сущности / изменение финансовой модели
v1.1	Hold / Release / Capture	Hold, HoldEvent; posted/held/available; reserve journal и atomic capture.
v1.1	Reversals & Refunds	TransferReversal, reversal LedgerTransaction; согласие получателя и достаточность средств.
v1.1	Transfer Fees	FeePolicy, FeeAssessment; platform revenue account, debit sender total incl fee.
v1.1	Statements & Analytics	StatementExport/aggregates; чтение posted ledger, CSV export, as-of balance.
v2	Recurring Transfers	RecurringTransfer, RecurringExecution, worker; unique scheduled occurrence + retries.
v2	Fraud Detection	FraudRule, FraudAssessment, FraudCase; allow/review/block, hold integration.
v2	KYC Simulation	KycCase, KycReview, verification status; ONLY fake identity test data.
v2.1	Multi-Currency & FX	Currency, ExchangeRate, FXQuote, FXConversion, FX clearing accounts; balancing per currency.

17. Open decisions and ADR recommendations

Decision	Предлагаемый выбор	Почему
UUID	UUIDv7 if tooling supports; otherwise UUIDv4	Упорядоченность vs простота.
Rejected transfer persistence	Persist business rejection + idempotency response for financial attempts	Наблюдаемость и повторяемость.
Wallet frozen behavior	Freeze both incoming and outgoing in MVP	Проще политика; уточнить до реализации.

Top-up operator	Dedicated sandbox role and seeded demo admin	Избегаем публичного mint endpoint.
Ledger enforcement	Controlled posting function + DB constraints/triggers	Защита от частичных проводок и произвольной правки.
Money representation	BIGINT minor units	Детерминированная точность.
Idempotency expiration	At least 24h for replay records; never remove business data	Баланс стоимости и надёжности.
User closure	Soft disable, no destructive delete of ledger	Неизменяемая финансовая история.

18. API backlog / implementation priority

P0: register/login, wallet create/read/balance, sandbox top-up, P2P transfer, transfer retrieval, ledger reconciliation, idempotency. P1: refresh/logout, audit, pagination, health, rate limits. P2: advanced metrics and clean demo. Прежде чем делать v1.1, доказать correctness через тесты гонок и инварианты.

19. Risks and mitigation

Риск	Митигация
Double spend	Row-level locks in deterministic order; concurrency tests.
Ledger/projection drift	One atomic transaction; periodic reconciliation test/query.
Replay after network timeout	Idempotency record in same transaction, stable response.
Incorrect liability accounting	Explicit debit/credit rules, fixtures and invariant tests.
Unsafe demo top-up	Sandbox-only operator role, rate/amount limits.
Scope creep	Strict MVP gate; versioned backlog for eight enhancements.
Accidental real-money assumptions	Clear sandbox banner; no financial provider integration or real funds.

20. Delivery documentation checklist

- README.md (English): value proposition, setup, stack, examples, limitations.
- docs/architecture.md + ADR-001 ledger approach; ADR-002 idempotency.
- docs/data-model.md with ERD and field definitions.
- docs/api.md or published OpenAPI + error catalogue.
- docs/invariants.md and concurrency/rollback demonstration.
- CI badge + GitHub Actions running unit/integration tests.
- Docker Compose including PostgreSQL + Redis, migrations, seed demo accounts.
- Synthetic data only. Never publish live secrets or personal identity documents.

End of PRD • Baseline v1.0 • FinCore Digital Wallet